

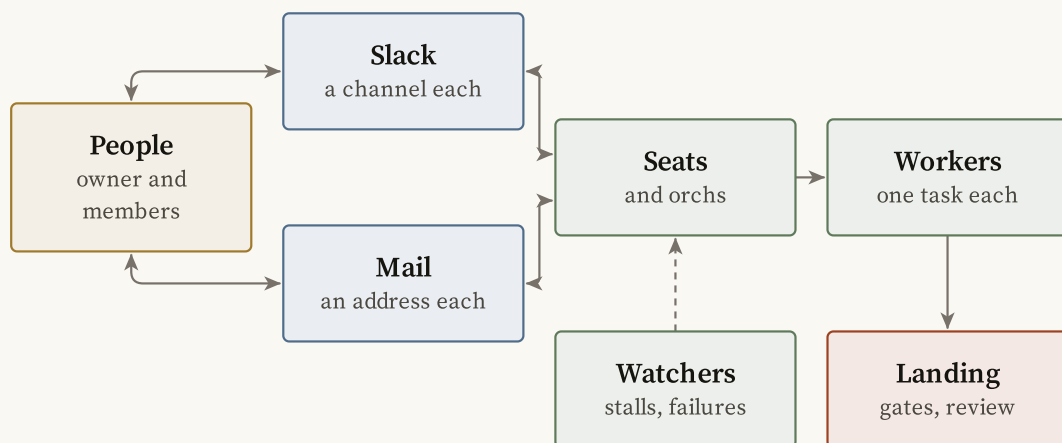
The agentic systems inside autobox

A home server run by Claude Code sessions, drawn one system at a time.

autobox is a headless home server. Nobody sits at it: its owner and a few other people talk to it by Slack from a phone, or by mail, and long-running agent sessions do the work. Every change still reaches the main branch through tests and a review. This document draws each agentic system on it, grouped by what it is for, from the core loop outward.

In the diagrams, sage boxes are agents, rust boxes are checks, slate boxes are ways in or out, and ochre is a person.

FIGURE 1 *The whole box*

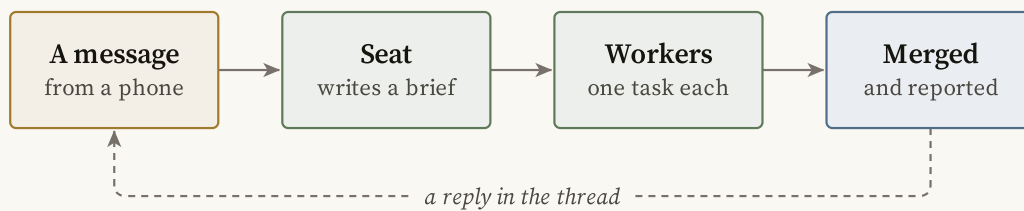


People write in by Slack or mail. Planning sessions split the work, workers do it, and a landing step decides what reaches main. Watchers wake a planning session when something stalls or keeps failing.

Getting work done

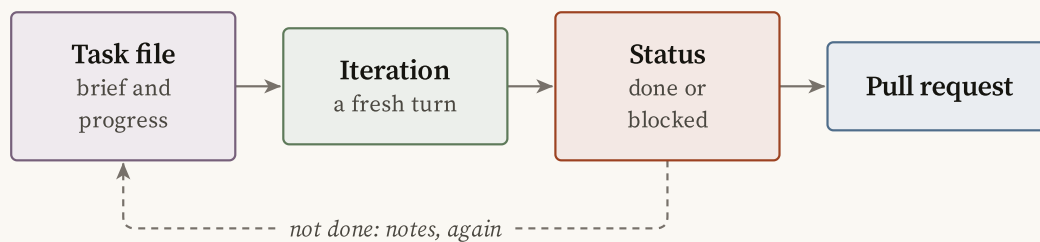
Four systems carry a request from a phone to merged code. They are built around one cost: what an agent reads on every turn. So long-lived sessions stay small, the heavy work runs in short loops that start fresh each turn, and nothing reaches main without passing the same gates.

FIGURE 2 *The planning seat*



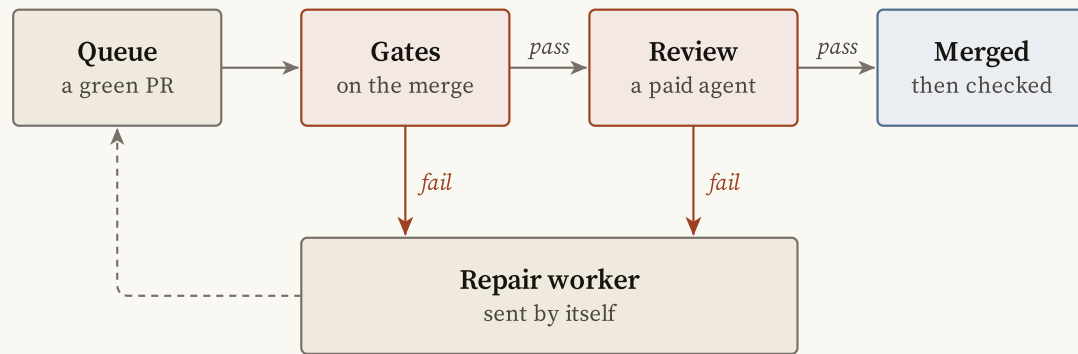
Each project has one long-lived agent, its planning seat. It takes a message from a phone, writes a short brief, sends out workers and reports back in the same thread. It plans and does not write the code itself, so its context stays small enough to last for days.

FIGURE 3 *The worker loop*



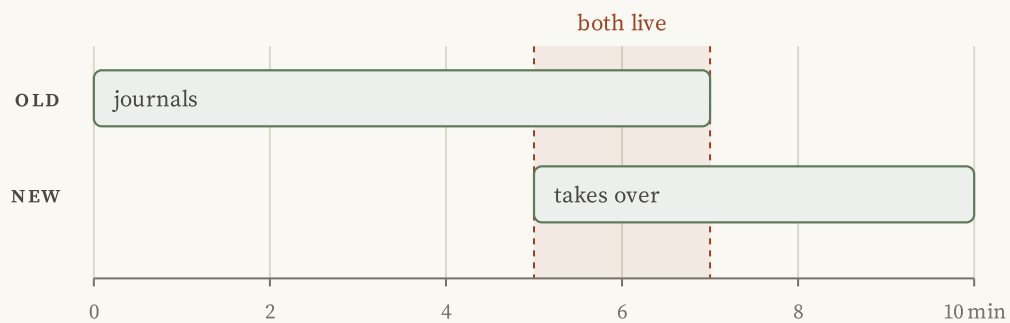
A worker is a headless loop with a spending cap, working in its own git worktree. Each turn starts fresh from the brief and its own progress notes, until it says done or blocked. Starting fresh keeps every turn cheap, and the notes file means a crash loses one turn, not the task.

FIGURE 4 *Landing and review*



This is the only way into main. The tests run on the merge result, not the branch, and a separate agent reviews the change. When a gate fails, it sends out one repair worker by itself, so a red build gets fixed without anyone asking.

FIGURE 5 *Handoff*

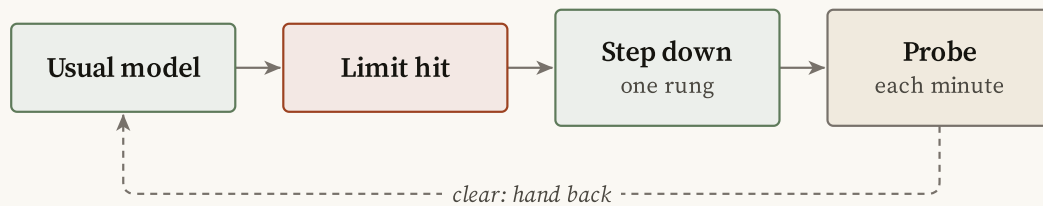


A session near its context limit starts a successor, which reads the journal and asks what it missed. Both run for a while, and then the old one retires. Handing over early is cheaper than carrying a huge context, and the overlap means no message falls between them.

Running unattended

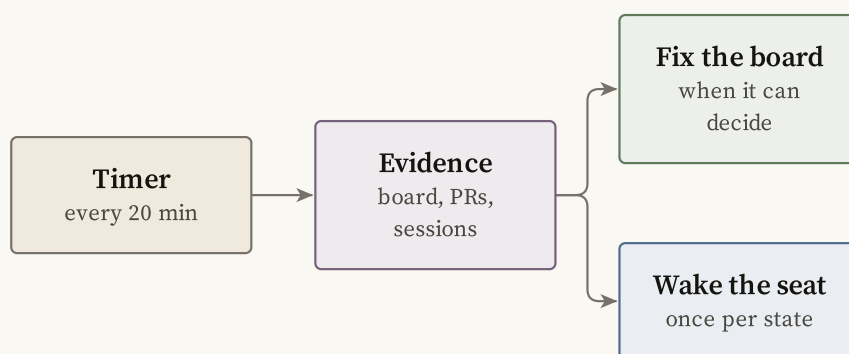
Nobody watches autobox, so these systems deal with what goes wrong while no one is there to see it. Each one either fixes the problem itself or wakes a planning session once, never over and over, because every wake-up costs money.

FIGURE 6 *Model switching*



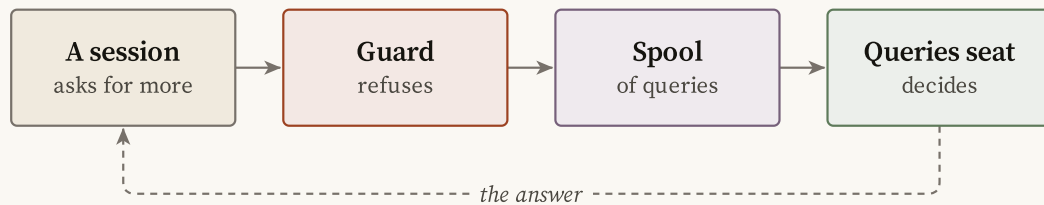
When a model hits its usage limit, every session on it steps down one model and checks each minute whether the limit has cleared. When it has, the session hands back to its usual model. Planning sessions never drop below Opus, because a weaker planner writes worse briefs for everyone downstream.

FIGURE 7 *The stall watcher*



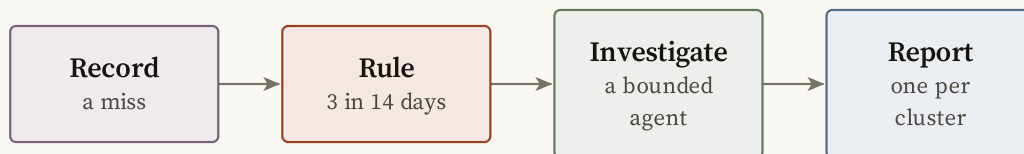
Every 20 minutes it reads the task board, the open pull requests and the live sessions. It fixes what it can decide on its own and wakes the planning session once about the rest. It runs on a timer, so no session has to stay awake just to check.

FIGURE 8 *The queries session*



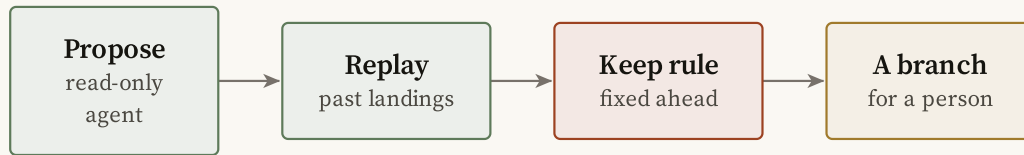
A guard checks what every session tries to run. When it refuses something, the refusal becomes a query, and a session kept for this decides it and answers. The session that was refused carries on with other work, and the owner only hears about the few cases that are really his to decide.

FIGURE 9 *Failure noticing*



Every time a session admits a miss, such as a wrong answer or a lost result, it records it. When the same kind recurs, three times in fourteen days, a read-only agent with a fixed budget looks into the cluster and writes one report. One mistake is noise, and a pattern is worth fixing.

FIGURE 10 *Self-improvement*

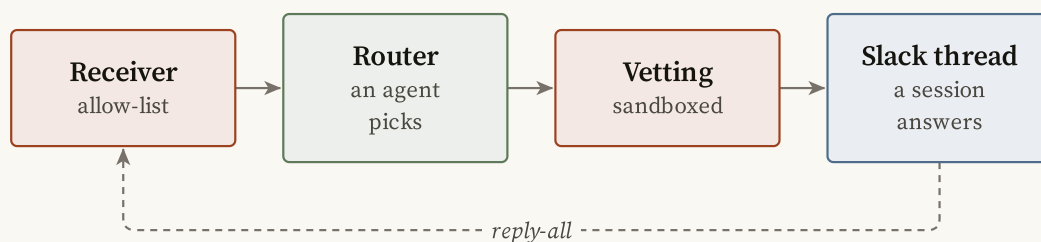


An agent proposes a change to the landing step. The change is replayed against past landings and kept only if a rule written in advance says it does better. Fixing the rule before the replay stops the agent from grading its own work generously.

Ways in

A person reaches autobox through Slack, mail or a workspace of their own. These systems route each message to the right session and keep what one person can do inside what they were given.

FIGURE 11 *The mail door*



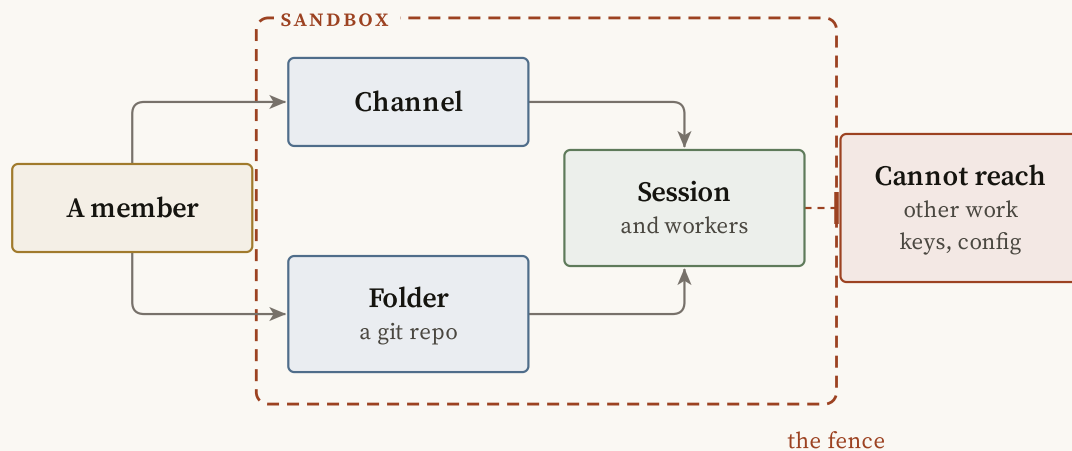
Mail to the box's own domain is checked against an allow-list, an agent picks which channel it belongs to, and it is vetted in a sandbox first. The session's answer goes back as a reply-all. Vetting comes before any session sees the text, since anyone can send mail.

FIGURE 12 *The ask channel*

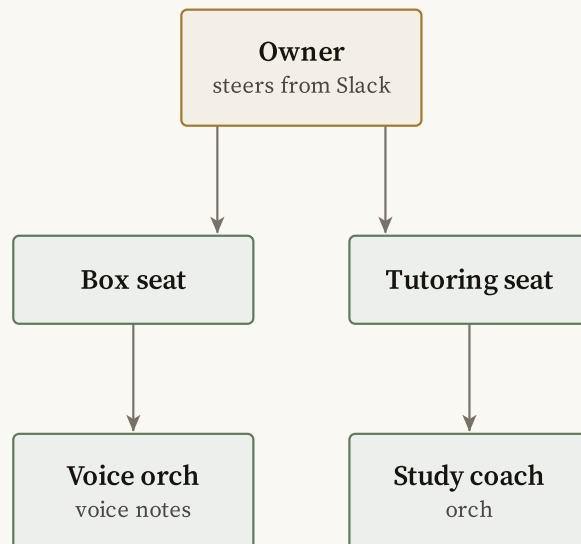


A quick question about a project goes to a one-shot agent on a small model, which answers from the project's written record and cites it. The planning session is never woken for it, so a status question costs very little.

FIGURE 13 *Member workspaces*



Each person the owner adds gets a channel, a git folder and a session of their own, fenced off from the rest of the box. The fence is a sandbox around the session rather than a line in its instructions, so it holds whatever the session is asked to do.

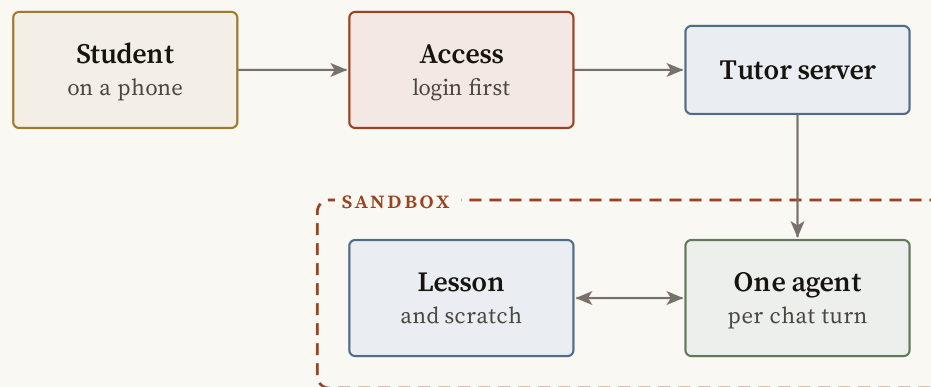
FIGURE 14 *Orchestrators*

A large project the owner steers gets its own orchestrating session and channel under its planning seat. Two run today: one for voice notes on the box, and a study coach inside a course-tutoring project. Keeping them apart stops one big project from filling the planning seat's context.

Agents inside projects

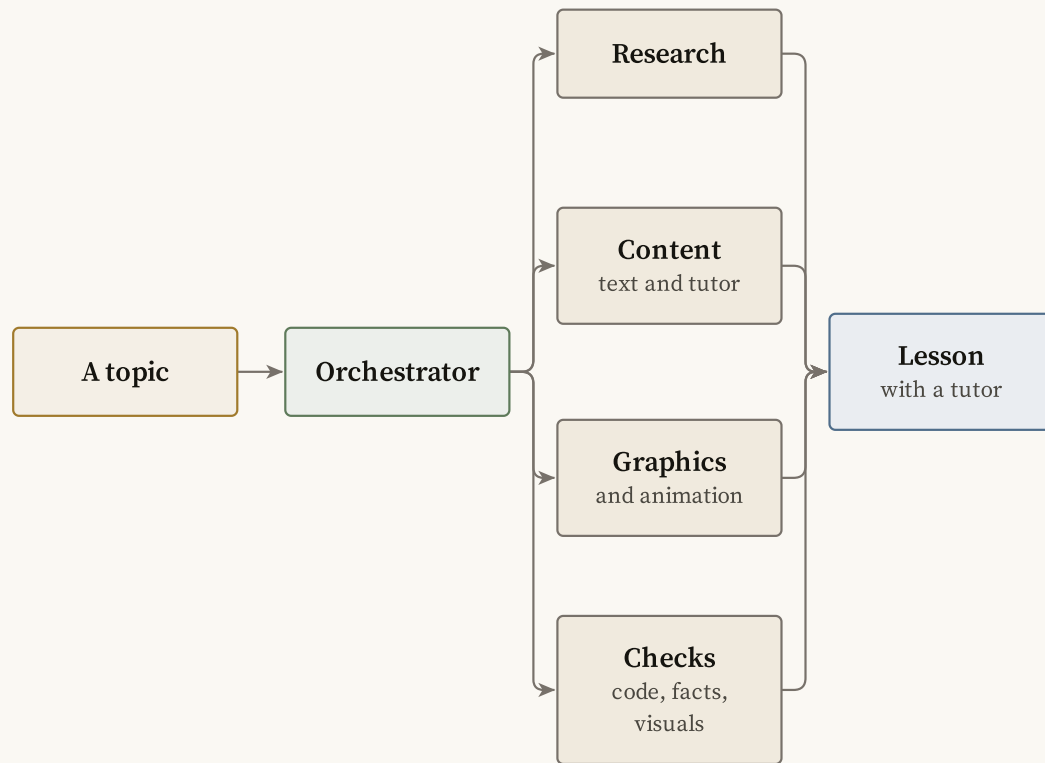
These systems belong to single projects rather than to the box. Two serve students, one does outside research, and one builds a large piece of work in steps.

FIGURE 15 *The hosted tutor*



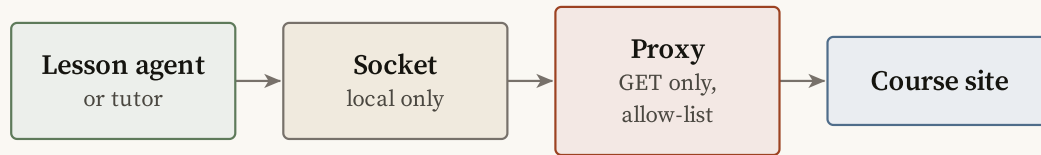
Students chat with a tutor beside each lesson in a course-tutoring project. Every turn is one agent in a fresh sandbox holding only that lesson and the student's scratch space. A fresh sandbox per turn means one student can never see another's work.

FIGURE 16 *The lesson generator*



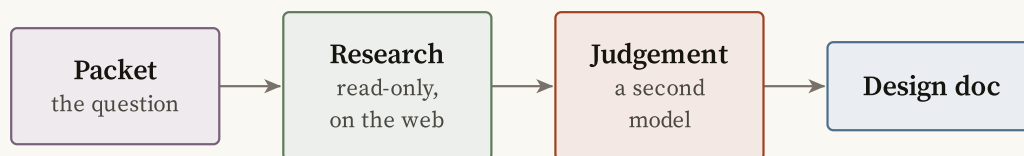
A skill turns a topic into an interactive lesson. An orchestrator hands the parts to specialist agents for research, writing and graphics, and a checking pass looks at the code, the facts and the visuals before it ships. Specialists with narrow jobs make fewer mistakes than one agent doing everything.

FIGURE 17 *Course page fetching*

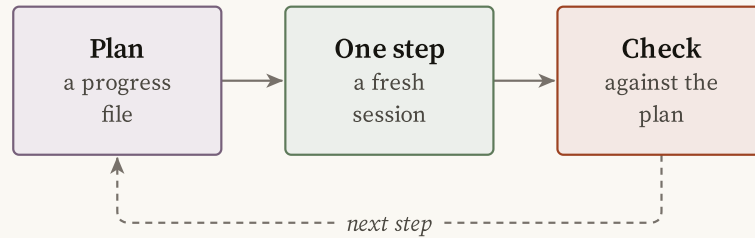


The lesson agents and the tutor fetch course pages through one local socket and a proxy that allows only reads, from sites on an allow-list. So an agent that is misled can read an allowed page but cannot post, change or reach anything else.

FIGURE 18 *Research runs*



Deep outside research runs read-only with web access, and a second model judges the result before it becomes a design document. The judge is a different model, so it does not share the first one's blind spots.

FIGURE 19 *A staged design build*

A circuit-design project builds its design skill one planned step at a time. Each step runs in a fresh, isolated session and is checked against the plan. A plan file that outlives every session lets a build take weeks without any one context growing.